CANDY
DIALOGUE ENGINE

Voiced Dialogue

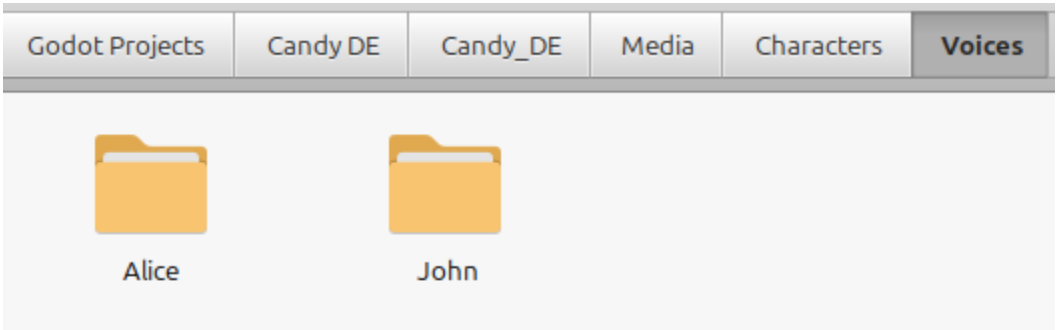
1.0.0

Voices dialogue is made remarkably easy in Candy DE: thanks to the Candy Dialogue Creator, you can assign a voice file to a spoken line in just a couple of clicks.

Folder Structure

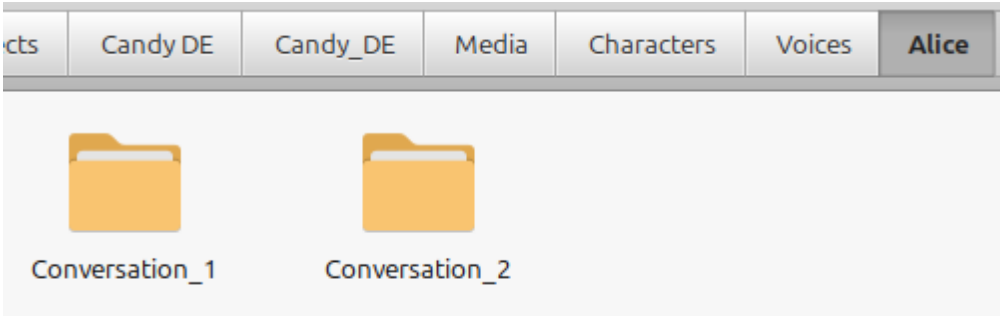
Before you can assign voice files to lines, you'll need to organize your voice files correctly inside the resources folders.

Assuming you're using the default folder setup, open **Candy_DE/Media/Characters/Voices**:



In the **Voices** folder, you must create a subfolder for every actor who has voiced lines.

Inside each **Actor** folder, you must create a subfolder for each Conversation (or at least those the actor has voiced lines for):



Finally, inside each **Conversation** folder, you must create a subfolder for each Block that has voiced lines for the actor:

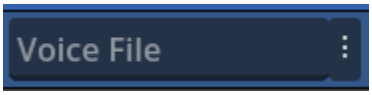
Finally, place the voice files inside these **Block** folders. The full setup:

Candy_DE/Media/Characters/Actor/Conversation/Block

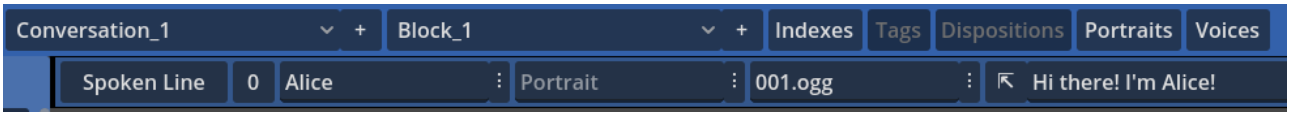
Assigning Files

Main UI

Once your voice files are in the correct **Block** folders, in the Candy Dialogue Creator, look at the "Voice File" field of a Spoken Line:



In that field, just add the correct voice file:



You can type the name of the file, but if you've setup your voice files and folders correctly, if you've provided a speaker reference for the line, and if you've provided the Candy Dialogue Creator with the path to your project, you can make things even easier:

Just click the (⋮) button next to the Voice File field, and a list of all voice files matching the speaker and the Block will appear! Just click to select the right one!

And if you're not sure which voice file is the correct one for the line, just hover any file in the list to listen to it!

As a little extra convenience, if you type a file's name, you don't need to write the extension (e.g. .ogg or .mp3) as long as Candy DE is configured to use the same extension as the default for voice files.

Default extensions are configured in **Candy_Properties.gd** under the Default File Extensions section:

```
#^ Default File Extensions:
#? The default extensions used by Candy DE when a media file name with no extension is provided.
#TODO: Edit extensions to match your preferences.
var default_portrait_extension = ".png"
var default_bust_extension = ".png"
var default_voice_extension = ".ogg"

var default_background_extension = ".png"
var default_image_extension = ".png"
var default_sound_extension = ".ogg"
var default_video_extension = ".ogv"
```

Writer UI

In the Candy Dialogue Creator's Writer UI, things are even more convenient:

Under the character portrait, you can assign a voice file to a similar Voice File field, and you even have controls to preview it: play, pause, stop, progress slider and volume slider!

Translations & Variants

There is no need to manually assign voice files for translations or other variants of a line:

1) Inside each **Block** folder where voice files are stored, create a subfolder for any variant, and name it the same as that variant.
2) Place variant voice files in their corresponding **variant** folders.
3) Make sure the variant voice files share the same name as their 'Default' counterpart voice files.

That's it. No need to assign the variant voice files to lines explicitly: if a spoken line has a voice file assigned for its Default variant, Candy DE will find the correct voice file for any variant, thanks to the fact they're named the same.

What if the Default variant doesn't have a voice file, but some variants have voice files?

No worries! Make sure all the variant voice files have the same name. Then, in the Voice File field, write that name just like if a Default voice file actually existed.

Candy DE won't crash: it will simply not play anything for the Default variant, since no such file exists!

Finally, it's worth noting that in the Writer UI, variants have their own Play, Pause and Stop buttons: these let you preview the variant voice files, if they exist. However, the progress and volume sliders are shared with the Default voice file: you can't play both at once anyway, they share the same AudioStreamPlayer.

Candy DE Settings

Now how does Candy DE actually handle voice files?

By default, it just plays them when it processes a line. But you can configure a few settings in **Candy_Engine.gd** under the "Settings" region:

No Voice Skipping

```
#^ Wait on voice playback:
#? Prevents the player from skipping voice playback, even if the line has finished writing.
#% true = prevent skipping voice
#% false = allow skipping voice
var await_voice_playback = false
```

If you don't want the player to be able to skip to the next line until voice playback is finished, set **await_voice_playback** to 'true'.

This also prevents auto-read mode from continuing to the next line until voice playback is finished. Note that auto-read will start counting down as soon as the line text is fully displayed, but if the timer expires and the voice file is still playing, it will wait for it to finish.

Synchronize Voice & Writing

```
#^ Synchronize writing speed to voice:
#? Adjust the writing speed to be timed with voice playback (for voiced lines).
#+ Only works if writing_speed > 0.
#% true = synchronize writing speed to voice
#% false = don't synchronize
var sync_write_speed_voice = false
```

If you use progressive writing (typewriter effect) you can make Candy DE synchronize the writing speed to the duration of the voice file. Just set **sync_write_speed_voice** to 'true', and it will automatically calculate the correct writing speed!

Disable Voice

```
#& Voiced Dialogues
#^ Allow voice:
#? Enable or disable voice playback for spoken lines:
var enable_voice = true
```

Finally, if you want to disable voice playback for spoken lines, set **enable_voice** to 'false'.

Note that this will not affect "Voice" dialogue mode: this would render Voice mode entirely pointless, as it doesn't display any text on screen.

But if for some reason you really want to have the option of disabling voice for "Voice" mode too, just open **Candy_Engine.gd**, go to the **display_line()** function, and in the match statement, navigate to "Voice".

There, find this block:

```
#@ Start voice file playback:
if voice_player.stream != null:
    voice_player.play()
    await wait_for_player_advance()
```

```
if voice_player.stream != null:
      voice_player.play()
      await wait_for_player_advance()
```

And change the if statement like this:

```
#@ Start voice file playback:
if enable_voice and voice_player.stream != null:
    voice_player.play()
    await wait_for_player_advance()
```

```
if enable_voice and voice_player.stream != null:
      voice_player.play()
      await wait_for_player_advance()
```

Remember to always keep notes of any changes you make to Candy_Engine.gd, so you can easily re-apply them after updates. For safety, make back-ups before modifying the code.

Text-To-Speech

A final option for voicing dialogues is to use a Text-To-Speech (TTS) tool.

Candy DE has built-in code to support such tools, but it's up to you to integrate one in your game, and write a small function so Candy DE can use it.

For more details, see the TTS guide.